

Moment 1: Orchestration Rule Authoring as the Design-Time Exercise of Human Governance in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 1 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the first of the two moments at which human governance is exercised in CKS systems — orchestration rule authoring at design time — as a standalone architectural commitment with independent operational content, separable from the direct-override moment (Moment 2) with which it composes.

Abstract

The CKS pattern's “human-governed” commitment is exercised at two moments: orchestration rule authoring at design time and direct override at intervention time (§3.3 of the source paper). The integrated treatment of those two moments is what makes the architecture's linear-cost commitment defensible, since neither moment scales with substrate size (§6.3). A separate note formalizes the joint commitment. This note formalizes the first moment, rule authoring, as having independent operational content that can be defended, implemented, and tested independently of the override moment. The motivation is concrete: rule authoring is widely conflated with adjacent activities — system configuration, prompt engineering, agent instruction-writing — that share surface features but operate at different architectural layers and produce different governance properties. The note states what rule authoring is in six operational components, identifies what it does not require, distinguishes it from the three adjacencies, names the cost properties that follow from it being a moment, enumerates the failure modes that violate it specifically, and provides an operational test for whether a given system implements rule authoring as CKS-coherent governance.

1. Why the rule-authoring moment needs to be formalized as standalone

The CKS pattern's human-governed commitment is exercised at two moments (§3.3): orchestration rule authoring and direct override. Together, the two moments carry the architecture's cost-scaling property — governance does not grow with substrate size because it is exercised at points in time distinct from substrate growth and from cell execution (§6.3). A separate derivation note formalizes the joint commitment as an authority architecture rather than a labor or review commitment.

The joint framing is correct as far as it goes, and this note does not contradict it. But it leaves Moment 1 underspecified in three directions, each of which corresponds to a misreading widespread in current practice. Read one way, rule authoring becomes ordinary system configuration — parameters set in a config file, deployment manifest, or environment management system — with the governance content quietly lost. Read another, it becomes prompt engineering — natural-language strings tuned for desired LLM behavior, held in prompt libraries outside the substrate — with the authority content lost. Read a third, it becomes one-shot pre-deployment setup — fixed at deployment, immutable thereafter — with the evolution content lost, since rules in fact get authored, deprecated, and re-authored over the substrate's lifetime as substrate content evolves.

Each misreading collapses an architectural commitment into a practice or workflow that has no architectural standing. Naming what rule authoring is, as a moment of governance exercise, is what keeps the commitment visible and what gives downstream implementers a precise specification of what their rule-authoring mechanism must satisfy independent of how cell execution and direct override are handled.

2. Rule authoring, defined precisely

In the CKS pattern, **rule authoring** is the act by which a human writes — or modifies, deprecates, or replaces — an orchestration rule that governs cell-level behavior. Orchestration rules are themselves substrate content (§2.3): persistent, addressable, traceable, and subject to the same three rights (inspect, modify, override) humans hold over any other substrate content. Rule authoring is therefore a special case of substrate write, distinguished by what is written: rules that determine how cells operate, not domain content the cells operate over.

The act has six operational components.

(a) The author is human. This is the load-bearing component. Rule authoring is a human exercise of authority. LLMs may draft rules, suggest rules, or critique rules; what they cannot do is commit rules to the substrate as authoritative without a human exercising authority in the act. The mechanism by which the human exercises authority varies by deployment — writing the rule directly, approving an LLM-drafted rule, editing a draft into final form — but what the architecture requires is that the commit is the human's act, not the LLM's.

(b) The rule is written to substrate content. Rules live in the substrate, not in cell configuration files, not in prompt files held outside the substrate, not in vendor-managed runtime settings, not in environment variables read at execution time. This is what makes rules governable on the same terms as other substrate content — inspectable, modifiable, overridable, traceable.

(c) The rule has provenance. Authorship, timestamp, and (where applicable) rationale are recorded as substrate metadata under the path retraceability commitment. A reader can determine, for any rule that governs cell behavior at any moment, who authored it under what authority and when.

(d) The rule takes effect once committed. Cell behavior governed by the rule operates under the new rule from the rule's commit forward. The architecture does not require pre-deployment validation,

simulation, or staged rollout as a precondition of effect; deployments may add such layers above the architectural commit, but the architectural act of authoring is complete at commit.

(e) The rule is amortized across cell executions. Once written, the rule applies to every cell execution within its scope that follows, without re-authoring per execution. This is the cost-scaling component: rule authoring is paid once per rule, not once per cell execution. A rule that governs ten thousand subsequent cell executions is authored once.

(f) The rule remains modifiable. Authoring is not a one-shot operation; rules are themselves subject to the three rights, which means they can be modified, deprecated, or replaced as substrate content evolves. The “design-time” framing does not mean rules are fixed at deployment. It means rule authoring is exercised at points in time distinct from cell execution.

The six components together define what rule authoring requires of the host environment, the substrate's representational form, and the act itself. Failing any one — even with the other five robustly satisfied — fails the moment architecturally.

3. What rule authoring does NOT require

The standalone treatment is not a maximalist treatment. Stating precisely what the moment does not require is what keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not require formal verification. Rules are human-authored; the architecture commits to neither their formal verifiability nor mechanical proof of their correctness. Whether a given rule does what its author intended is outside the architecture's scope, beyond the path-retraceability commitment that the rule's effect can be traced.

It does not require comprehensiveness. Rules cover what they cover; situations not covered by any rule are handled by direct override (Moment 2) or by humans operating outside cells. The architecture does not require rules to be exhaustive.

It does not require natural-language form. Rules can be written in natural language, structured DSL, code, or any other form the host environment supports. What the architecture requires is that rules are substrate content humans can author and inspect, not that they take any particular syntactic form.

It does not require pre-execution validation. A rule committed to the substrate takes effect on cell executions that follow. Deployments may layer validation, testing, simulation, or staged rollout on top; those layers are deployment choices, not architectural preconditions of the moment.

It does not require single authorship. Multiple humans may co-author a rule, edit each other's drafts, or layer authorship over time. What the architecture requires is that the commit is recorded with provenance identifying the human or humans who exercised authority.

4. What rule authoring is NOT

Three adjacent activities are commonly conflated with rule authoring. Each is a real and reasonable practice in some other architecture; naming what rule authoring is not is what prevents the misreading.

Not system configuration. System configuration in conventional software engineering is the work of setting parameters, environment variables, feature flags, or deployment settings that determine how an application runs. Configuration is typically held in configuration files, environment management systems, or deployment manifests — formats and locations that are not substrate content. Rule authoring in CKS is architecturally distinct: rules are substrate content, with authorship and provenance, governable on the same terms as the rest of the substrate. A deployment can have rich system configuration alongside rule authoring; what it cannot do is treat configuration as a substitute for rules, because configuration is not subject to the three rights humans hold over substrate content.

Not prompt engineering. Prompt engineering is the practice of designing the natural-language inputs that produce desired LLM behavior. Prompts in CKS may be part of how a rule is expressed, but the rule itself is not the prompt — it is substrate content that includes whatever prompt material the rule references, with all the substrate-level commitments (governance, provenance, modifiability). A prompt held outside the substrate, applied at cell execution time without being substrate content, is not a rule, regardless of how careful or sophisticated the prompt design. The architectural content of rule authoring is in the substrate commit, not in the prompt design.

Not agent instruction-writing. Agent frameworks typically allow humans to write instructions that determine how agents behave — system prompts, persona definitions, tool descriptions, planning templates. These instructions structurally resemble orchestration rules but operate at a different layer: instructions configure the agent (a coordination-mechanism-layer object); rules govern the cell (a coordination-knowledge-layer object). Rules in CKS are not agent instructions, and agent instructions are not rules, even when a particular cell happens to be implemented through an agent framework. The architectural commitment is to rules as substrate content; the agent framework, if used, is an adjacent component composed with the substrate, not a substitute for it.

5. The cost properties that follow from rule authoring being a moment

The cost-axis content of rule authoring follows from §2 and ties the moment-axis decomposition to the architecture's linear-cost commitment (§6.3 of the source paper; cost decomposition formalized in the linear-cost derivation note, Li, 25 April 2026). Four properties follow.

Once per rule, not once per cell execution. Rule authoring cost is paid at the moment of authoring and amortizes across every cell execution within the rule's scope that follows. A rule that governs ten thousand subsequent cell executions imposes its authoring cost once. This is what makes the moment a moment in the architecturally meaningful sense: it is exercised at a point in time distinct from the moments at which the rule's effect is felt.

Grows with rule variety, not with substrate size. In the three-dimensional cost decomposition the linear-cost note formalizes (Dimension A: substrate size; Dimension B: rule variety; Dimension C: intervention frequency), rule authoring sits squarely on Dimension B. A deployment with a hundred rules pays roughly a hundred times the authoring cost of a deployment with one rule, regardless of how

large the substrate grows. A deployment whose substrate grows to one million elements but whose rule count remains constant pays no additional rule-authoring cost as the substrate grows.

Grows with rule complexity, not with cell complexity. A simple rule that governs complex cells has low authoring cost; a complex rule that governs simple cells has higher authoring cost. The architecture supports both patterns; the cost profile follows from the deployment's choice of rule design, not from how much work the cells perform under the rule.

Re-authoring is per-modification, not per-execution. Modifying or deprecating an existing rule pays per-modification cost. Deployments that revise rules frequently pay per-revision cost; deployments that author rules once and rarely revise pay essentially one-time cost. In either case, the cost is paid at the moment of authoring or re-authoring, not at the moment of cell execution under the rule.

None of these costs scales with substrate size. This is the cost-axis content of why rule authoring is a moment of governance: governance happens at points in time independent of substrate growth and independent of cell execution volume. The moment-axis decomposition of the human-governed commitment, of which rule authoring is the first half, is what makes the linear-cost commitment defensible.

6. Failure modes that violate the rule-authoring moment

A system can fail rule authoring specifically, even when it satisfies other governance components. Six failure modes name the most common ways this happens.

(a) LLM-committed rules without human authority. When an LLM drafts a rule and the rule takes effect as substrate content without a human exercising authority in the commit, the architectural content of rule authoring is violated. The LLM may draft; the human must commit.

(b) Rules held outside the substrate. When orchestration rules live in cell configuration files, vendor-managed runtime settings, prompt libraries held alongside but not within the substrate, or environment variables read at execution time, the rules escape governance. Humans may have access to these representations, but the rules are not subject to the three rights as substrate content, which means they are not governed in the architectural sense.

(c) Rule authoring without provenance. When rule commits do not record authorship, timestamp, and (where applicable) rationale, the path-retraceability commitment is violated at the rule layer. Rules become effective without being traceable to who authored them under what authority.

(d) Rule changes that require re-authoring per cell execution. When a rule's effect cannot be amortized across executions — because the rule must be re-committed, re-validated, or re-applied per cell execution — the cost-amortization property is violated, and the moment-axis decomposition collapses. A construct that requires per-execution re-application is architecturally an inline configuration, not a rule.

(e) Rules architecturally fixed at deployment. When the architecture treats rule authoring as a one-shot pre-deployment operation, with no architectural support for modifying, deprecating, or

replacing rules during operation, the design-time framing has been over-collapsed. Rules are design-time in the sense that authoring is exercised at points distinct from cell execution; they are not design-time in the sense of being immutable post-deployment.

(f) Rules that gate the moment behind process as architectural precondition. When rule authoring requires multi-stage approval, formal review, or pre-commit validation as architectural preconditions — rather than as deployment-layer additions — the no-justification-as-precondition property of governance authority is violated for the rule-authoring case. The architecture commits to humans being able to commit rules on their authority; deployments may layer process above this without making the layer architectural.

A system that exhibits any of (a)–(f) does not implement rule authoring in the architectural sense, even if it supports rule-like configuration in some other form, and naming the failure precisely is what allows downstream remediation.

7. Operational test

A system implements the rule-authoring moment specifically if and only if all of the following are true at all times during the substrate's existence:

1. Orchestration rules are substrate content — addressable, inspectable, modifiable, traceable — not held in configuration or runtime layers separate from the substrate.
2. Rule commits are exercised by humans, with provenance (authorship, timestamp, and where applicable rationale) recorded as substrate metadata.
3. Once committed, rules take effect on cell executions within their scope from the commit forward, without architectural preconditions of validation, approval, or simulation.
4. The architectural cost of rule authoring is paid once per rule, not once per cell execution; the rule amortizes across executions within scope.
5. Rules remain modifiable, deprecatable, and replaceable after initial authoring, with each modification recorded with provenance.

A system that fails any of (1)–(5) does not implement the rule-authoring moment in the architectural sense, even if it supports rule-like configuration in some other form. Such a system may be useful for other purposes but is not CKS-coherent on this axis of governance, and downstream work that relies on its rule-authoring guarantees should be scoped accordingly.

8. Why naming this moment as standalone matters

Implementations that conflate rule authoring with system configuration produce systems where rules escape governance, with consequences for traceability and accountability that surface only when an audit, investigation, or override action requires reading the rule history that the configuration layer never recorded. Implementations that conflate rule authoring with prompt engineering produce systems where the architectural commitment to rules-as-substrate-content is replaced by practices — prompt

versioning, A/B testing, prompt files held alongside source code — that have no architectural standing and no path through the substrate's governance machinery. Implementations that conflate rule authoring with one-shot pre-deployment configuration produce systems where rule evolution is architecturally unsupported, even when rules are in fact revised over time, because the architecture has no language for the revisions and the revisions therefore happen outside it.

Naming the moment as standalone — with the six components in §2, the cost properties in §5, and the test in §7 — gives downstream implementers a precise specification of what their rule-authoring mechanism must satisfy, independent of how cell execution and direct override are handled. The moment-axis decomposition of governance — rule authoring at design time, direct override at intervention time — is what makes the cost-scaling property defensible: governance happens at moments distinct from substrate growth and from cell execution, and rule authoring is the first of those moments. Subsequent work that implements, extends, or argues against the rule-authoring component of CKS governance should use “rule authoring” in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Moment 1: Orchestration Rule Authoring as the Design-Time Exercise of Human Governance in the Coordination Knowledge Substrate Pattern*. 1 May 2026. ORCID: 0009-0004-8065-3235.